MSBA 7028: Deep Learning

Lecture 2:Linear Neural Network

Xiao Lei

Logistics

- Google form for participation is on moodle
- Fill it out on the same day
- Always let me know your questions so that we can slow down

Reading

- Ch. 3 & 4 of d2l

Outline

- Linear algebra (Ch. 2.3)
- Linear regression (Ch. 3.1)
- Basic optimization
- Code for regression (3.5)
- Softmax regression (Ch. 4)

Linear algebra

Easy concepts

- Scalar
- Vector
- Matrix
- Tensor

Vectors

- Simple operations

$$c = a + b \quad \text{where } c_i = a_i + b_i$$
$$c = \alpha \cdot b \quad \text{where } c_i = \alpha b_i$$
$$c = \sin a \quad \text{where } c_i = \sin a_i$$

- We always assume that a vector is a n-row 1-column matrix.

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, x^T = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

Vectors

- Dot product

$$a^\top b = \sum_i a_i b_i$$

- We will also call it inner product, and use $< a, b >$ to refer it.

Matrices

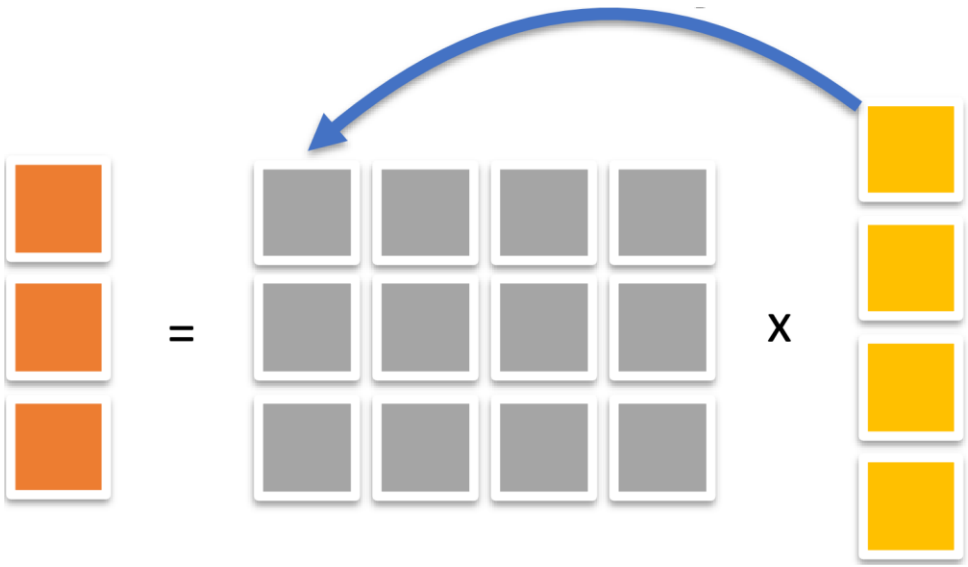
- Simple operations

$$C = A + B \qquad \text{where } C_{ij} = A_{ij} + B_{ij}$$
$$C = \alpha \cdot B \qquad \text{where } C_{ij} = \alpha B_{ij}$$
$$C = \sin A \qquad \text{where } C_{ij} = \sin A_{ij}$$

Matrices

- Multiplications (matrix vector)

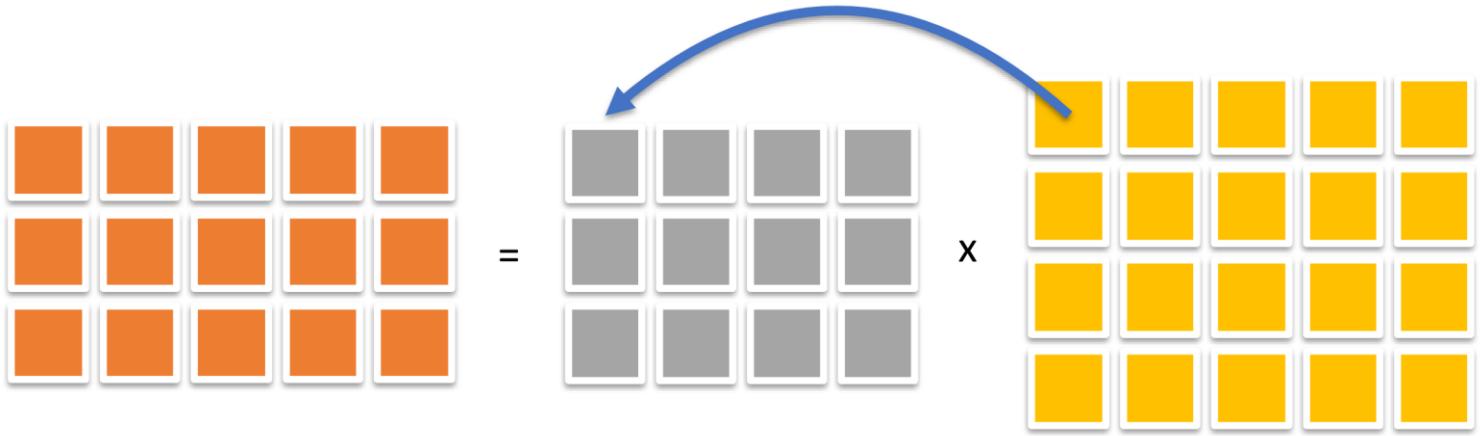
$$c = Ab \text{ where } c_i = \sum_j A_{ij} b_j$$



Matrices

- Multiplications (matrix matrix)

$$C = AB \text{ where } C_{ik} = \sum_j A_{ij} B_{jk}$$



GPU and linear algebra

GPUs love matrices and vectors
(they have many processors)

N-dimensional Array Examples: Tensor

- N-dimensional array, short for ndarray, is the main data structure for machine learning and neural networks

0-d (scalar)	1-d (vector)	2-d (matrix)
1.0	[1.0, 2.7, 3.4]	[[1.0, 2.7, 3.4] [5.0, 0.2, 4.6] [4.3, 8.5, 0.2]]
A class label	A feature vector	A feature matrix

Tensor Examples

3-d

```
[[[0.1, 2.7, 3.4]
  [5.0, 0.2, 4.6]
  [4.3, 8.5, 0.2]]
 [[3.2, 5.7, 3.4]
  [5.4, 6.2, 3.2]
  [4.1, 3.5, 6.2]]]
```

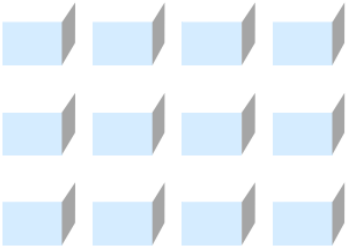
A RGB image (width x height x channels)

4-d

```
[[[[. . .
   . . .
   . . .]]]
```

A batch of RGB images (batch-size x width x height x channels)

5-d



```
[[[[. . .
   . . .
   . . .]]]
```

A batch of videos (batch-size x time x width x height x channels)

Tensors

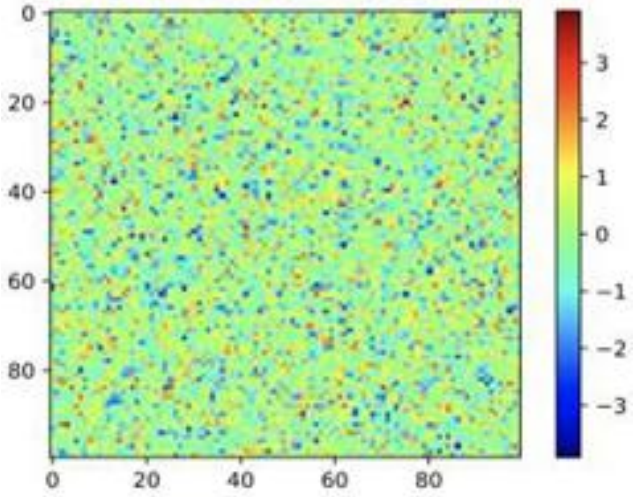
- Each tensor has the following attributes:
  - A shape, e.g. 3-by-4 matrix
  - Data type for each element, e.g. float
  - Element values, e.g all 0s, or random values

Generate a tensor in pytorch

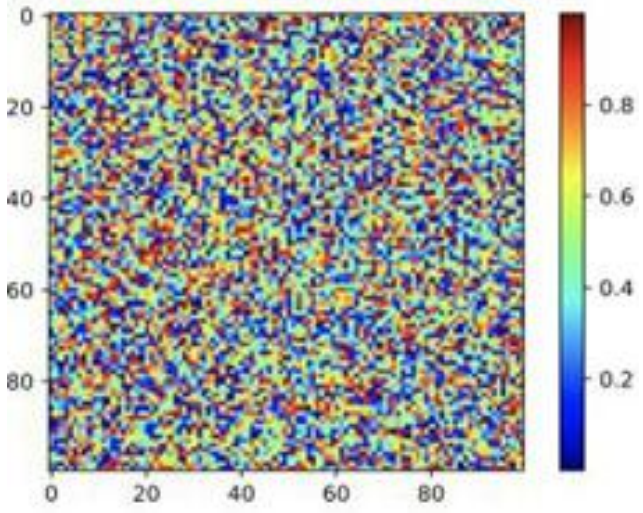
```
torch.zeros((2, 3, 4))
torch.normal((2, 3, 4))
```

100-by-100 matrix with elements generated from

A normal distribution



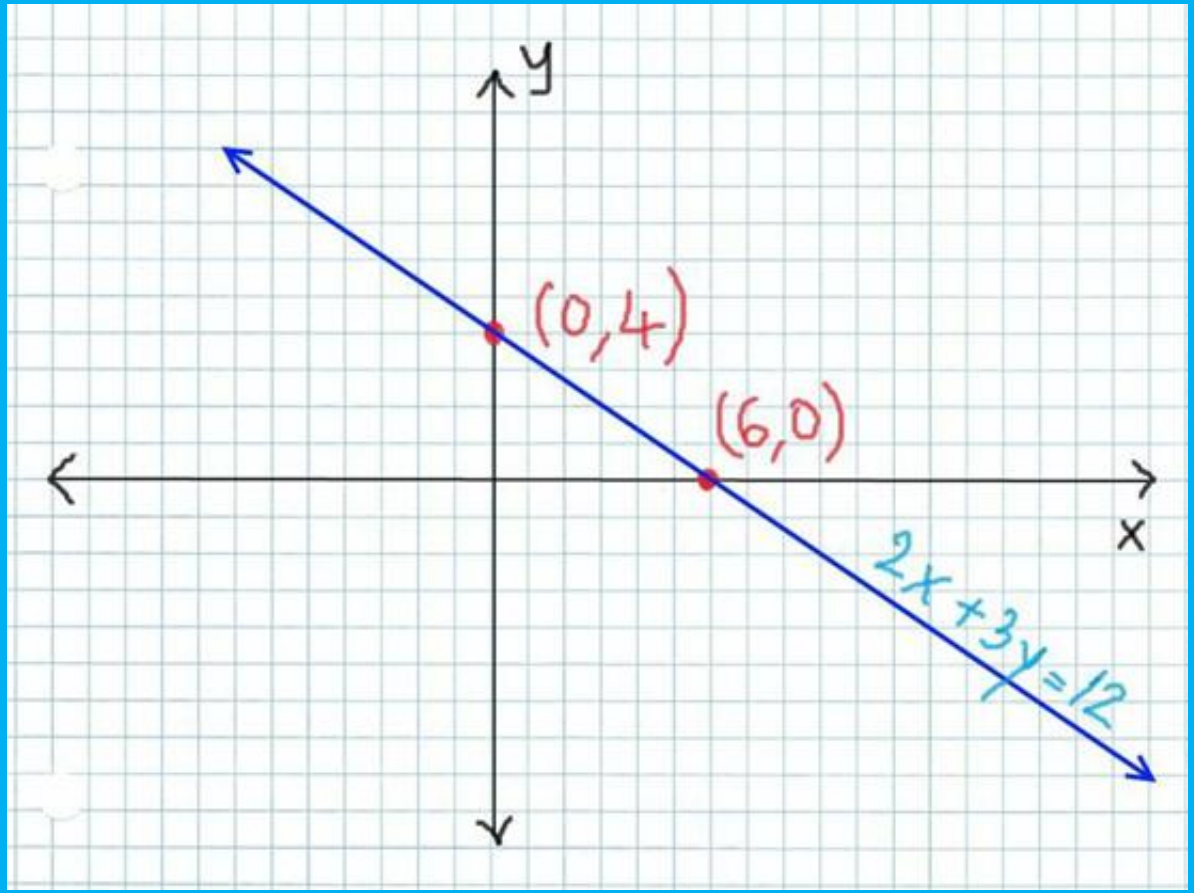
A uniform distribution



Code

- Code for data manipulation and linear algebra (Ch. 2.1, 2.3)

Linear regression



House Buying 101

Listing price from agent

$5,498,000	7	5	4,865 Sq. Ft.
Price	Beds	Baths	$1130 / Sq. Ft.

Redfin Estimate: $5,390,037 On Redfin: 15 days

Predicted sale price

Virtual Tour

- Branded Virtual Tour
- Virtual Tour (External Link)

Parking Information

- Garage (Minimum): 2
- Garage (Maximum): 2
- Parking Description: Attached Garage, On Street
- Garage Spaces: 2

Multi-Unit Information

- # of Stories: 2

School Information

- Elementary School: El Carmelo Elementa
- Elementary School District: Palo Alto Uni
- Middle School: Jane Lathrop Stanford Mi
- High School: Palo Alto High
- High School District: Palo Alto Unified

Interior Features

Bedroom Information

- # of Bedrooms (Minimum): 7
- # of Bedrooms (Maximum): 7

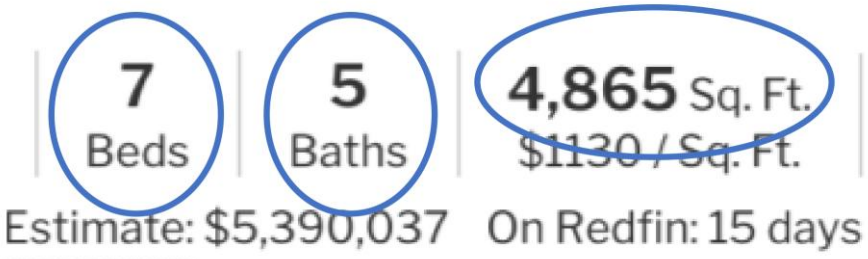
- Kitchen Description: Countertop (Granite Dishwasher, Garbage Disposal, Hood Ove Island with Sink, Microwave, Oven Range

A Simplified Model

- Assumption 1

The key factors impacting the prices are

#Beds, #Baths, Living Sqft, denoted by x_1, x_2, x_3



- Assumption 2

The sale price is a weighted sum over the key factors

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

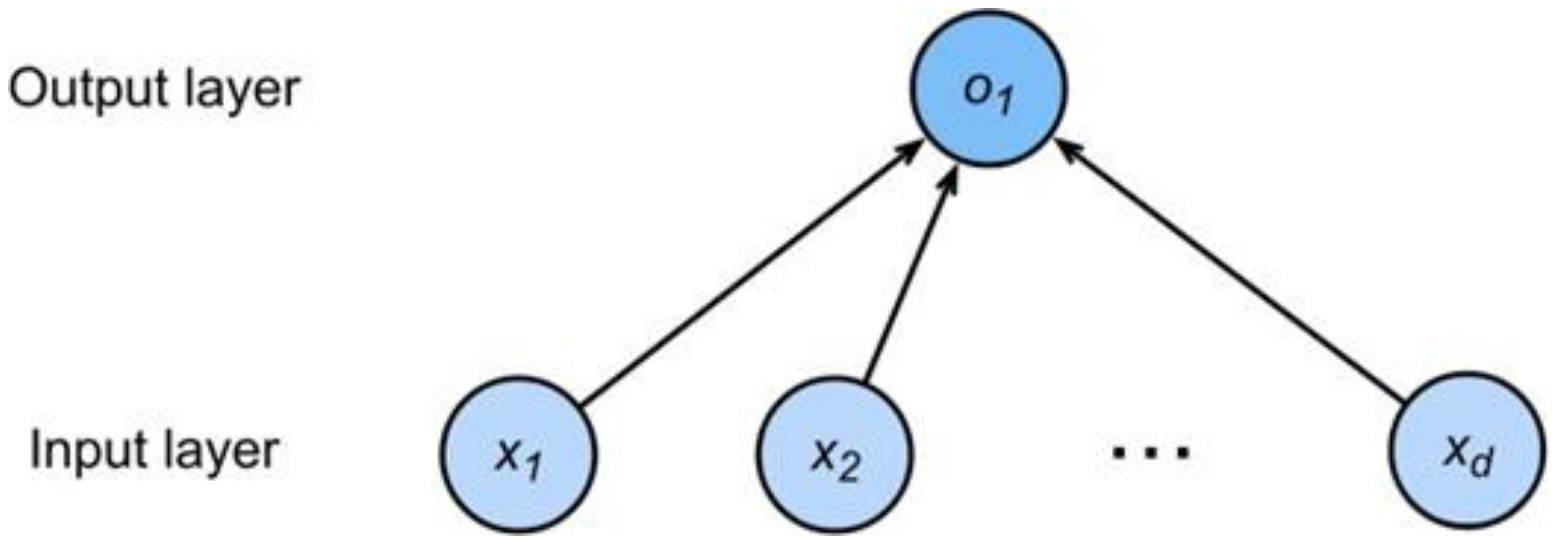
Weights and bias are determined later

Linear Model

- Given *d*-dimensional inputs $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$
- Linear model has a *d*-dimensional weight and a bias
  - $\mathbf{w} = [w_1, w_2, \ldots, w_d]^T, b$
- The output is a weighted sum of the inputs
  - $y = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$

Vectorized version $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$

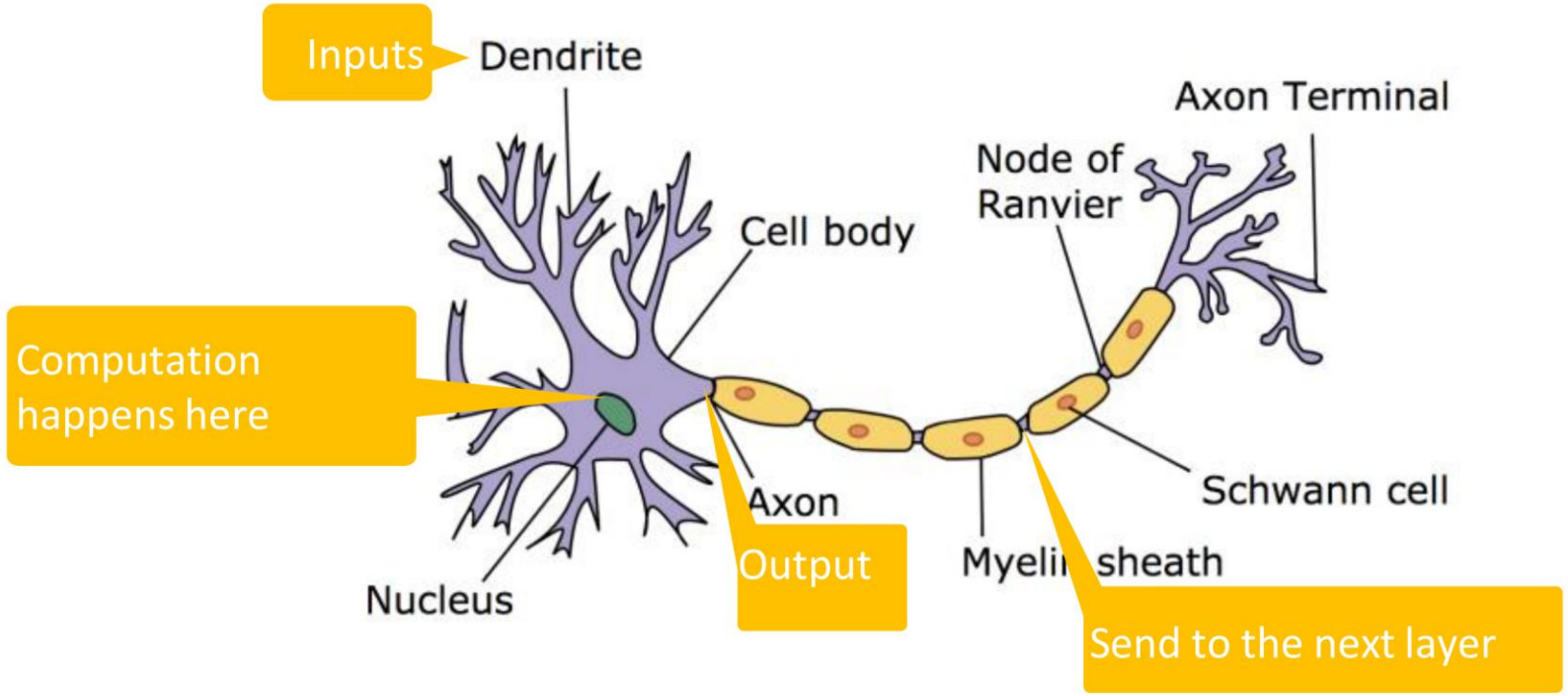
Linear Model as a Single-layer Neural Network



We can stack multiple layers to get deep neural networks

Neural Networks Derive from Neuroscience

The real neuron



Code for linear layer

- nn.linear(d,1): d is the number of feature, 1 is the number of output
- nn.LazyLinear(1): only specify the number of output

Measure Estimation Quality

- Compare the true value vs the estimated value
- Real sale price vs estimated house price
- Let y be the true value, and $\hat{y}$ be the estimated value, we can compare the loss

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

It is called squared loss

Question: why not other losses?

Training Data

- Collect multiple data points to fit parameters

Houses sold in the last 6 months

- It is called the training data
- The more the better
- Assume *n* examples

$$\mathbf{X} = \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n{}^T \qquad \mathbf{y} = y_1, y_2, \ldots, y_n{}^T$$

Each sample is a vector

Learn Parameters

- Training loss

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b)^2 = \frac{1}{n} \| \mathbf{y} - \mathbf{X}\mathbf{w} - b \|^2$$

- Minimize loss to learn parameters

$$\mathbf{w}^*, \mathbf{b}^* = \underset{\mathbf{w}, b}{\operatorname{argmin}}\ \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b)$$

Note: The computation volume depends on the sample size.

Closed-form Solution

- Add bias into weights by $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}]$ $\quad \mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} \| \mathbf{y} - \mathbf{X}\mathbf{w} \|^2 \qquad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

- Loss is convex, so the optimal solutions make the derivative to be 0

$$\frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = 0$$

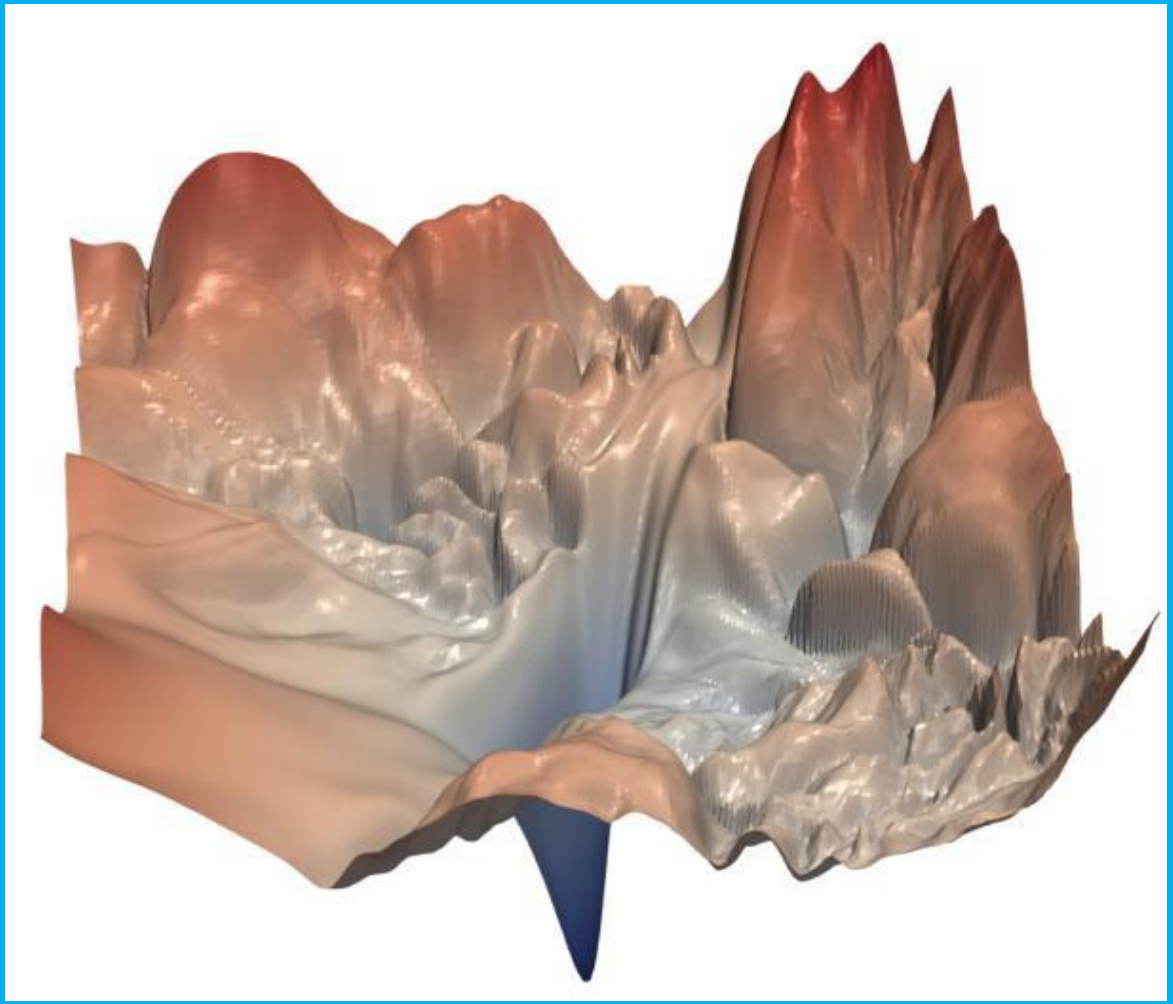
$$\Leftrightarrow \frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X} = 0$$

$$\Leftrightarrow \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Summary of linear regression

- Weighted average plus bias
- Squared loss measures the predicted and true response
- There is closed form solution
- Can be viewed as single layer NN

Basic optimization



Training is an optimization task

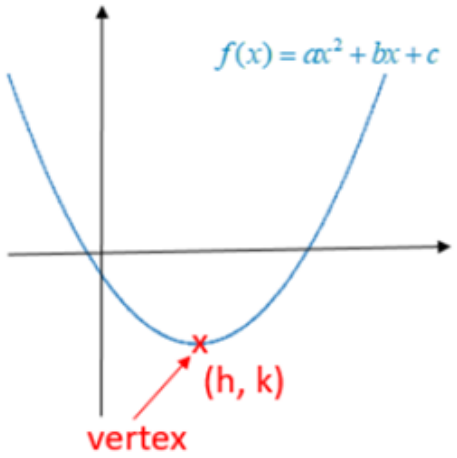
- When we train a machine learning model, we are solving an optimization problem

$$\min_{\mathrm{w}} l(w, X, y)$$

Where l is the loss function, w is the model parameter. The core is to search a set of parameters w to minimize the loss (in-sample inaccuracy)

- For linear regression with squared loss, the parameter is w, b, and the loss function is $(wX + b - y)^2$.

- Minimize a quadradic function is easy:



Training is an optimization task

- When we train a machine learning model, we are solving an optimization problem

$$\min_{\mathrm{w}} l(w, X, y)$$

Where l is the loss function, w is the model parameter. The core is to search a set of parameters w to minimize the loss (in-sample inaccuracy)

- For linear regression with squared loss, the parameter is w, b, and the loss function is $(wX + b - y)^2$.

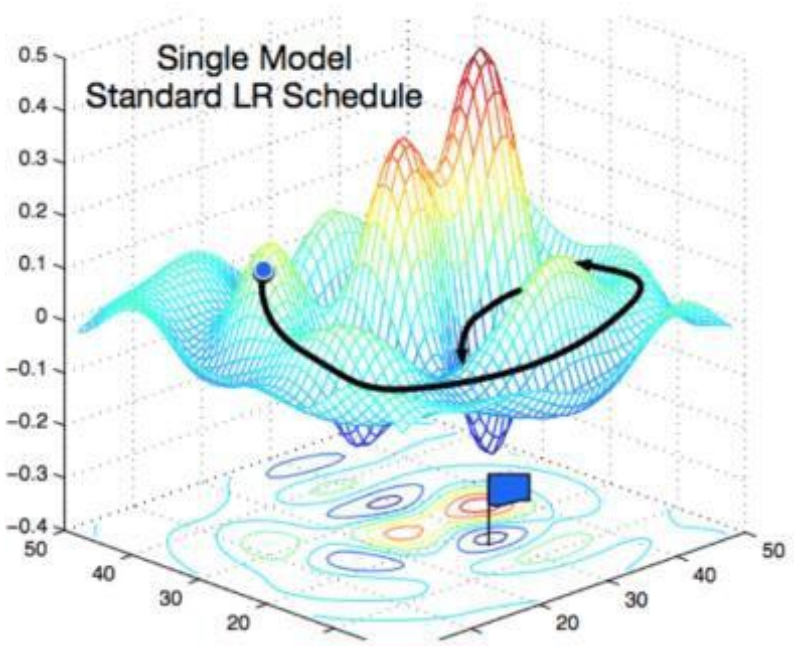
- Solving an optimization problem is not easy. In most cases, we cannot find the best w (also called global optima) in a reasonable time.

Optimization methods

- Exhaustive search
- Random search (genetic algorithms)
- Analytical solution
- Model-based search (e.g. Bayesian optimization)
- Neural nets usually use *gradient-based search*

- Why bother? Training is not easy anymore with big data, usually takes hours or even days. Efficiency matters.

Gradient Descent Analogy

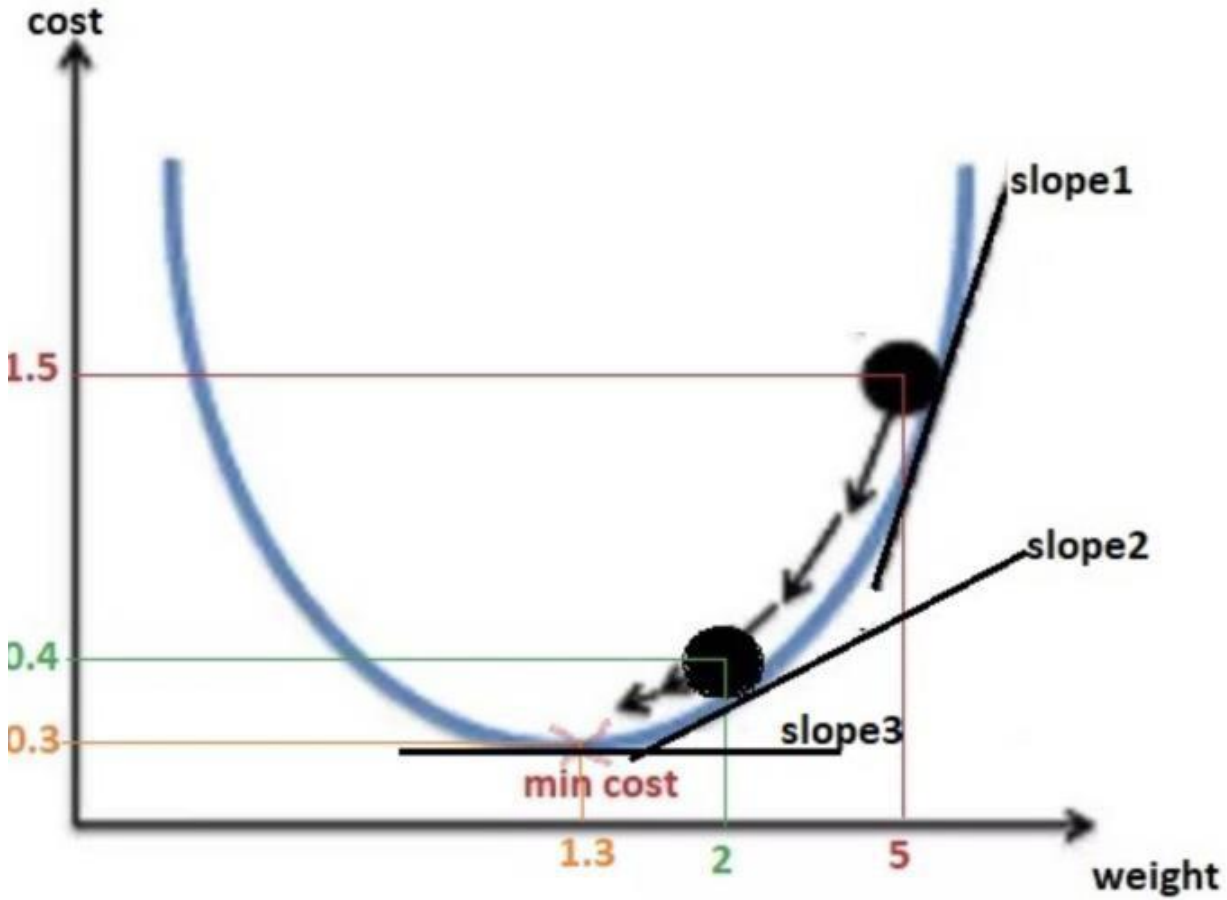


Walking man image is CC0 1.0 public domain

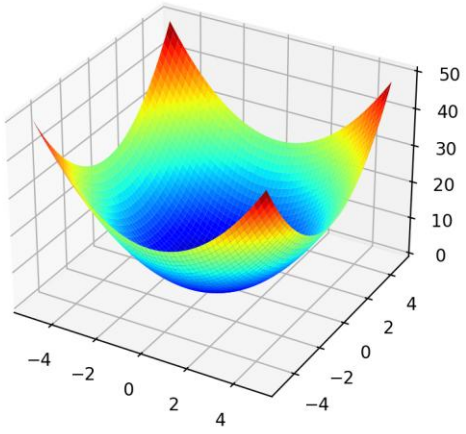
Gradient

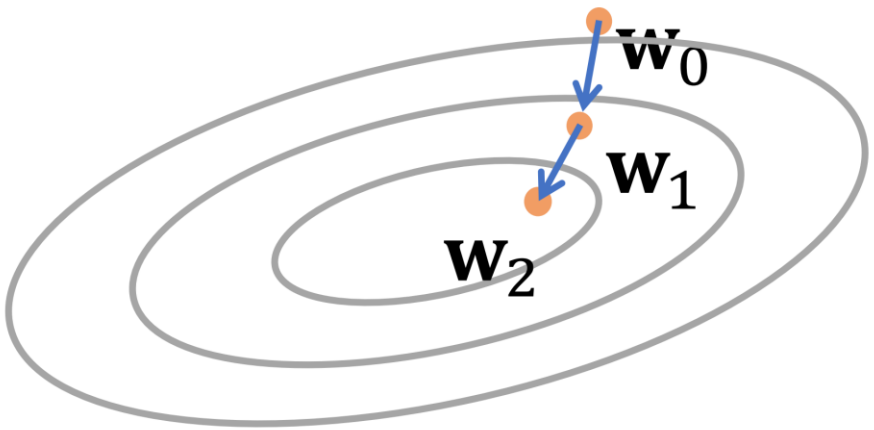
- A gradient measures how much the output of a function changes if you change the inputs a little bit.
- ‘Slope of a function’
- In mathematical terms, a gradient is a partial derivative with respect to its inputs.
- The direction of steepest descent is the negative gradient

Gradient Descent Algorithm



Two dimensional example





Suppose at a point $\boldsymbol{w_0} = (w_{01}, w_{02})$:

$$\frac{\partial l(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w_0})}{\partial w_{01}} = 1$$

$$\frac{\partial l(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w_0})}{\partial w_{02}} = 10$$

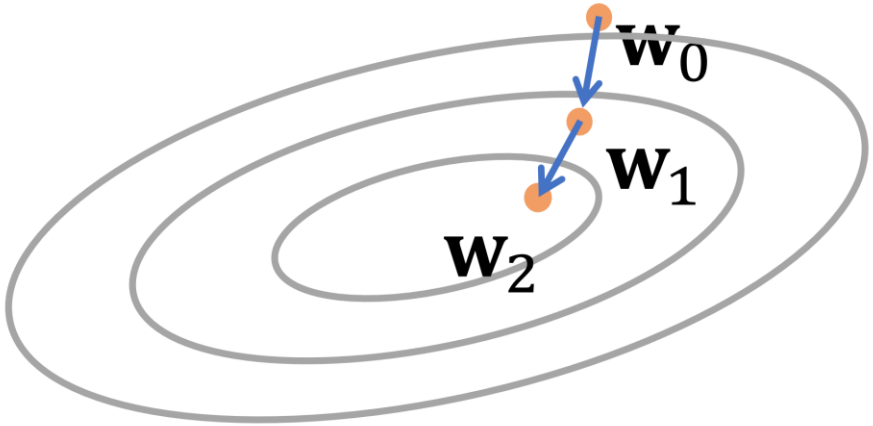
Decreasing w_{01} by 1 reduce the loss by 1
Decreasing w_{02} by 1 reduce the loss by 10

I should decrease both w_{01}, w_{02}, but I will reduce 0.1 unit of w_{01} while reducing 1 unit of w_{02}.

Gradient Descent

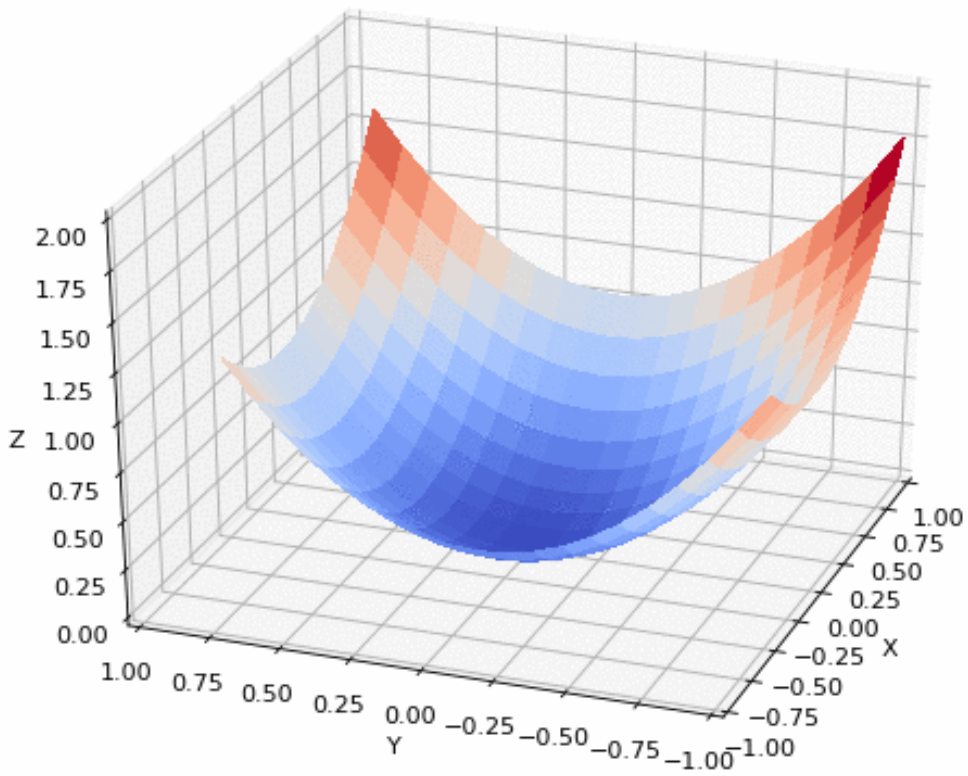
- Choose a staring point $\mathbf{w}_0$
- Repeat to update the weight *t=1,2,3*

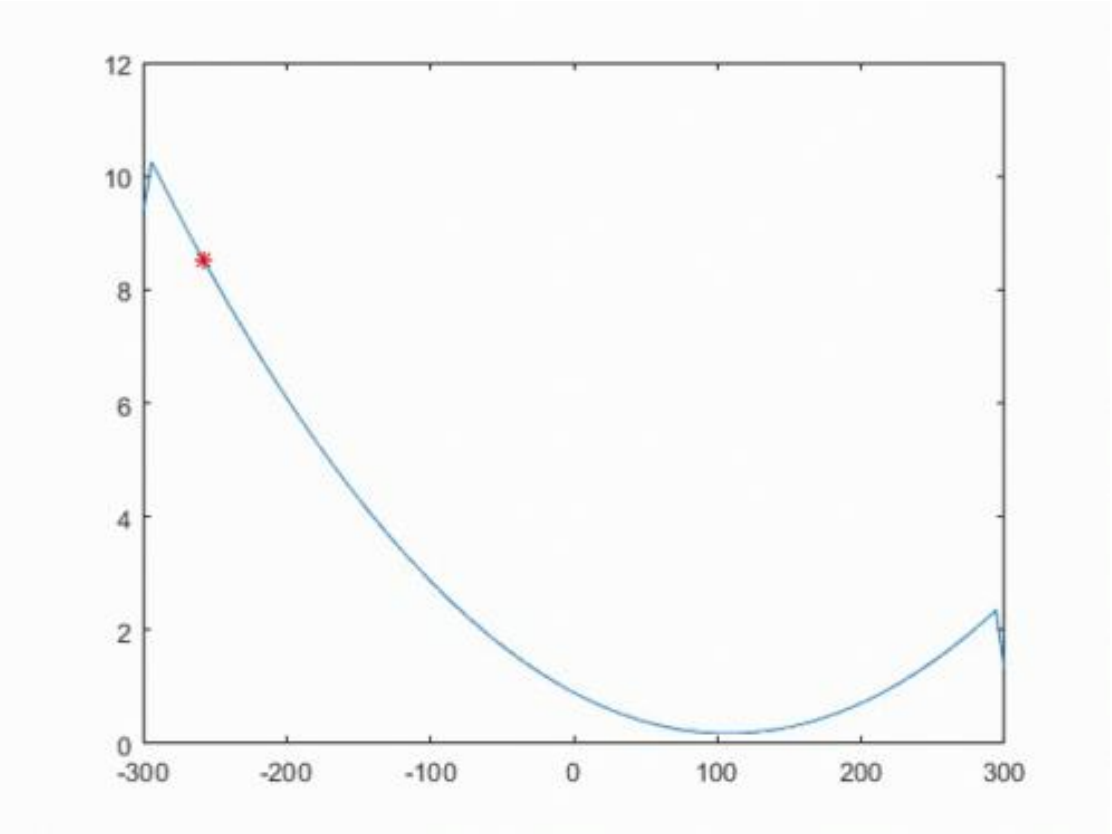
$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$



- Gradient $\frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$: a direction that increases the value

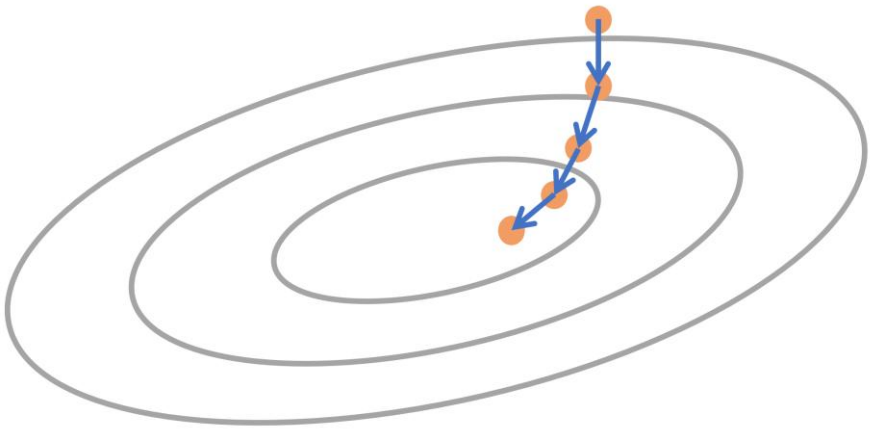
- Learning rate η: a hyper-parameter specifies the step length


Z
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Y
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
X
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

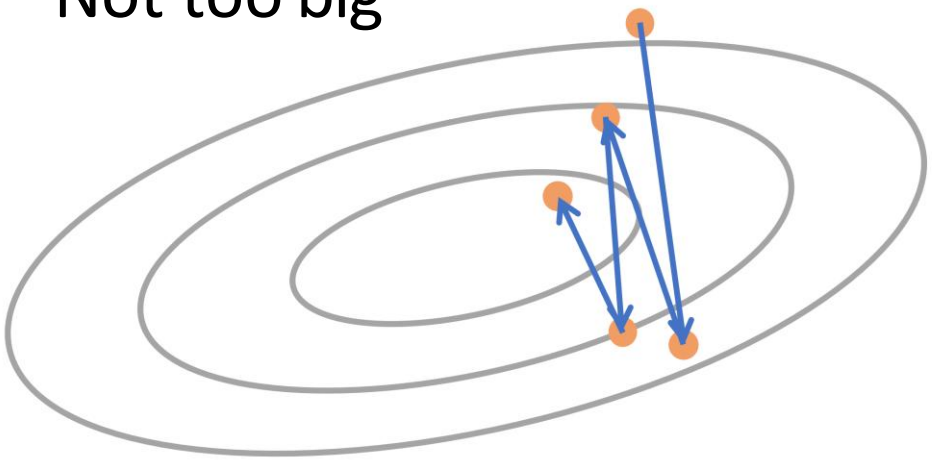

12
10
8
6
4
2
0
-300
-200
-100
0
100
200
300

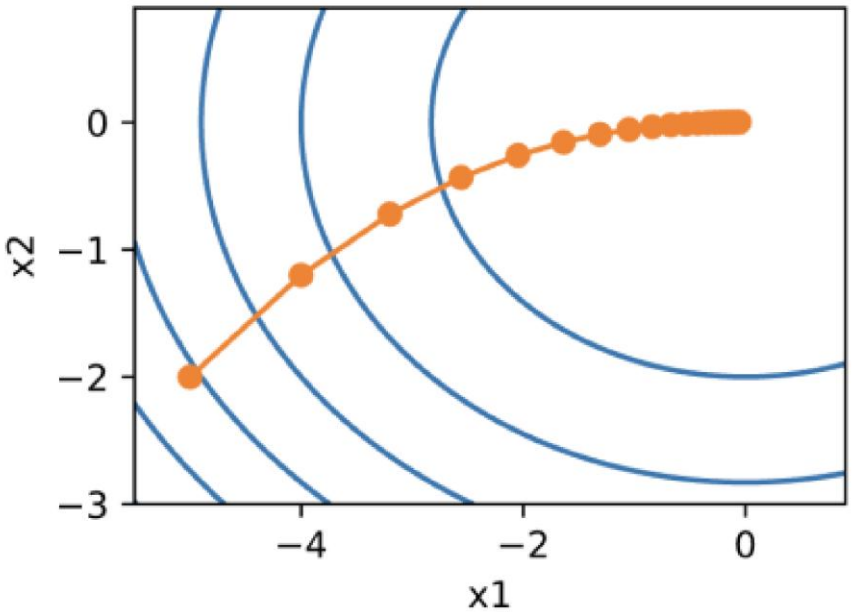
Choose a Learning Rate

Not too small



Not too big





Summary

- Problem: estimate a real value
- Model: $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$
- Loss: square loss $\ell(y, \hat{y}) = (y - \hat{y})^2$
- Learn by gradient descent
- Choose a starting point
- Repeat
  - ·Compute gradient
  - ·Update parameters